

# Demystifying Gradient Descent with Momentum

*A Unified Guide: Bridging PyTorch, Textbooks, and Physics*

## The Source of the Confusion

When studying optimization across different contexts—Deep Learning frameworks (like PyTorch), standard Machine Learning textbooks, or classical convex optimization papers—you will inevitably encounter wildly different equations for Momentum.

**Do not panic!** They are all mathematically identical. Standard gradient descent is like a hiker taking blind steps based only on the immediate slope. Momentum turns the hiker into a heavy boulder: it builds up speed, remembers its past trajectory, and plows through small bumps. The differences in the equations simply lie in *which physical quantity* the intermediate memory variable is storing: the raw gradient, the physical step, or the absolute position.

### 1. The Deep Learning View (Tracking Raw Gradients)

Used in Deep Learning frameworks. Here, the velocity term ( $v_t$ ) tracks an exponentially weighted moving average of the **raw gradients** only. The learning rate ( $\eta$ ) and the negative sign (to step downhill) are applied *after* the accumulation.

$$\begin{aligned} v_t &= \beta v_{t-1} + \nabla \mathcal{L}(\theta_t) && \text{(Accumulate raw gradients)} \\ \theta_{t+1} &= \theta_t - \eta v_t && \text{(Scale by learning rate and step)} \end{aligned}$$

*Intuition:* “Gather all the slopes I’ve seen recently, then take a step of size  $\eta$  in the opposite direction.”

### 2. The Textbook View (Tracking the Actual Step)

Common in standard Machine Learning courses. Here, the memory term ( $m_t$ ) tracks the **actual physical parameter updates**. The learning rate ( $\eta$ ) and the negative direction are baked into  $m_t$  from the very beginning.

$$\begin{aligned} m_t &= \beta m_{t-1} - \eta \nabla \mathcal{L}(\theta_t) && \text{(Accumulate actual physical steps)} \\ \theta_{t+1} &= \theta_t + m_t && \text{(Add the step directly to the weights)} \end{aligned}$$

*Intuition:* “Take a fraction  $\beta$  of my exact previous physical step, and merge my new gradient-based step into it.”

### 3. The Physics View (Polyak's Heavy Ball)

This classical mathematical view skips creating a separate memory variable altogether. It recognizes that "velocity" is literally just the physical distance traveled in the previous step:  $(\theta_t - \theta_{t-1})$ .

$$\theta_{t+1} = \theta_t \underbrace{-\eta \nabla \mathcal{L}(\theta_t)}_{\text{Current Gradient Step}} + \underbrace{\beta(\theta_t - \theta_{t-1})}_{\text{Momentum from last step}}$$

*Intuition:* "Where was I moving last time? Keep going that way a bit ( $\beta$ ), but also adapt to the current slope ( $\eta$ )."

## The Ironclad Proof of Equivalence

To see why these are all the exact same algorithm, we can algebraically convert Formulations 1 and 2 into the physics-based Formulation 3. Let the current gradient be  $g_t = \nabla \mathcal{L}(\theta_t)$ .

### Proof: Deep Learning View $\rightarrow$ Physics View

Start with the Deep Learning update rule:  $\theta_{t+1} = \theta_t - \eta v_t$ . This tells us that the physical step taken is perfectly defined as  $-\eta v_t$ . Therefore:

$$\theta_{t+1} - \theta_t = -\eta v_t \implies \text{And for the previous step: } \theta_t - \theta_{t-1} = -\eta v_{t-1} \quad (1)$$

Now, take the gradient accumulation equation and multiply both sides by  $-\eta$ :

$$\begin{aligned} v_t &= \beta v_{t-1} + g_t \\ (-\eta v_t) &= \beta(-\eta v_{t-1}) - \eta g_t \end{aligned}$$

Substitute our physical steps from Equation (1) into the left and right sides:

$$\begin{aligned} (\theta_{t+1} - \theta_t) &= \beta(\theta_t - \theta_{t-1}) - \eta g_t \\ \theta_{t+1} &= \theta_t - \eta g_t + \beta(\theta_t - \theta_{t-1}) \quad \blacksquare \quad (\text{Matches the Physics View exactly!}) \end{aligned}$$

### Proof: Textbook View $\rightarrow$ Physics View

This one is even easier. We know  $\theta_{t+1} = \theta_t + m_t$ , meaning  $m_t = \theta_{t+1} - \theta_t$  and  $m_{t-1} = \theta_t - \theta_{t-1}$ . Substitute  $m_{t-1}$  directly into the accumulation equation:

$$\begin{aligned} m_t &= \beta m_{t-1} - \eta g_t \\ m_t &= \beta(\theta_t - \theta_{t-1}) - \eta g_t \\ \theta_{t+1} &= \theta_t + \underbrace{\beta(\theta_t - \theta_{t-1}) - \eta g_t}_{\text{This is } m_t} \quad \blacksquare \quad (\text{Matches the Physics View exactly!}) \end{aligned}$$

## Seeing is Believing: A Numerical Example

If algebra isn't your preferred language, let's trace the actual numbers. Let's optimize a simple 1D quadratic loss landscape:  $\mathcal{L}(\theta) = \frac{1}{2}\theta^2$ . The gradient is simply  $g_t = \theta_t$ .

Let's set **Learning Rate**  $\eta = 0.1$ , **Momentum**  $\beta = 0.5$ , and initialize at  $\theta_0 = 10$ .

For the very first step, assume initial velocity is zero:  $v_0 = 0, m_0 = 0, \theta_{-1} = 10$ .

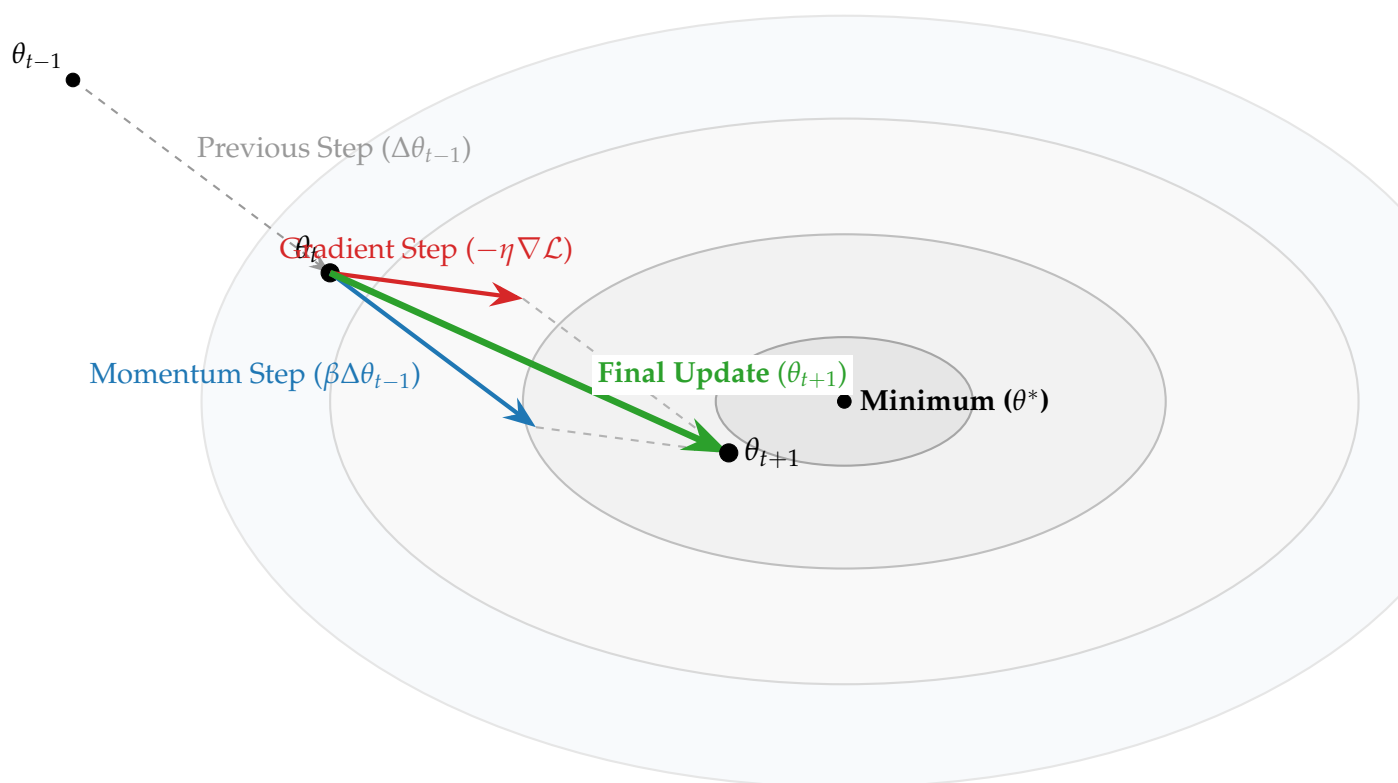
| Step          | Formulation 1 (DL)                                                                                            | Formulation 2 (Textbook)                                                                                            | Formulation 3 (Physics)                                                               |
|---------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Step 1</b> | $\text{Grad } g_0 = 10$<br>$v_1 = 0.5(0) + 10 = \mathbf{10}$<br>$\theta_1 = 10 - 0.1(10) = \mathbf{9}$        | $\text{Grad } g_0 = 10$<br>$m_1 = 0.5(0) - 0.1(10) = \mathbf{-1}$<br>$\theta_1 = 10 + (-1) = \mathbf{9}$            | $\text{Grad } g_0 = 10$<br>$\theta_1 = 10 - 0.1(10) + 0.5(0) = \mathbf{9}$            |
| <b>Step 2</b> | $\text{Grad is now } g_1 = 9$<br>$v_2 = 0.5(10) + 9 = \mathbf{14}$<br>$\theta_2 = 9 - 0.1(14) = \mathbf{7.6}$ | $\text{Grad is now } g_1 = 9$<br>$m_2 = 0.5(-1) - 0.1(9) = \mathbf{-1.4}$<br>$\theta_2 = 9 + (-1.4) = \mathbf{7.6}$ | $\text{Grad is now } g_1 = 9$<br>$\theta_2 = 9 - 0.1(9) + 0.5(9 - 10) = \mathbf{7.6}$ |

No matter how it is coded in the backend, the computer calculates the exact same trajectory!

## Microscopic View: The Parallelogram of Forces

Geometrically, momentum is pure vector addition. It is the combination of your **Current Gradient Step** (where the slope wants to pull you) and your **Momentum Step** (where your past inertia pushes you).

Because  $\beta < 1$ , the momentum vector is essentially a "shrunk down" ghost of your previous physical step. The final destination is achieved by drawing a perfect parallelogram using these two vectors.



## The Golden Takeaways

- **PyTorch / Deep Learning:** If a paper uses  $v_t$  to track just the **raw gradient**, remember that you must explicitly multiply by  $\eta$  and subtract it from your weights at the end.
- **Textbooks:** If a paper uses  $m_t$  to track the **negative gradient with  $\eta$  included**, it's already a physical displacement. You can simply add  $m_t$  directly to your weights!
- **Physics / Heavy Ball:** If you see  $(\theta_t - \theta_{t-1})$ , recognize this as the literal **physical definition of momentum**: continuing forward using your actual physical inertia from the last step.